DOCUMENTACIÓN TÉCNICA

Sistema de Especies Municipales

Auditoría, arquitectura, modelo de datos y guía de despliegue

Alcaldía Municipal de Santa Ana Norte

Unidad de Tesorería

Versión 1.0 · 25 de octubre de 2026

Repositorio ProyectoServicioSocial · rama master

Control del documento

Campo	Detalle
Documento	Documentación técnica y auditoría del Sistema de Especies Municipales
Versión	1.0
Fecha	25/10/2026
Alcance	Código fuente (`app`, `routes`, `config`, `resources/views`, `public`), migraciones, seeders, datos de carga inicial, dependencias (`composer.json`, `composer.lock`, `package.json`), configuración de entorno y esquema de la base de datos en funcionamiento.
Estado analizado	Rama `master`, commit `01c20d1` (sin cambios pendientes), base `db_proyecto_renta` con datos de noviembre de 2025 a junio de 2026.
Público	Equipo de desarrollo y mantenimiento, personal de TI que despliega el sistema y responsables técnicos de la Unidad de Tesorería.

Convenciones: los nombres de archivos, clases, métodos, rutas y columnas se escriben en `fuente monoespaciada`. Las rutas de archivo son relativas a la raíz del proyecto. Los importes monetarios están en dólares estadounidenses (USD).

Contenido

1. Resumen Ejecutivo y Propósito del Proyecto 5
1.1 Qué hace la aplicación 5
1.2 Problema que resuelve 5
1.3 Propuesta de valor técnica 5
1.4 Stack tecnológico 6
2. Arquitectura del Sistema y Patrones de Diseño 7
2.1 Arquitectura general 7
2.2 Patrones de diseño implementados 7
3. Estructura de Directorios y Organización del Código 9
4. Módulos y Lógica de Negocio 11
4.1 Mapa de módulos 11
4.2 Modelo de dominio de especies 11
4.3 Motor de inventario (App\Services\Inventario) 12
4.4 Flujos de negocio 15
4.5 Reportes PDF 16
4.6 Constancias de retención y reporte de consultas 17
4.7 Usuarios, roles y paneles 18
4.8 Carga inicial de datos (ETL) 18
5. Base de Datos y Modelo de Datos 19
5.1 Motor y características generales 19
5.2 Diagrama entidad-relación 19
5.3 Relaciones Eloquent 19
5.4 Diccionario de datos 20
5.5 Claves foráneas 26
5.6 Índices 27
5.7 Migraciones 27
5.8 Seeders y volumen de datos 28
6. APIs, Endpoints e Integraciones Externas 29
6.1 Estilo de interfaz 29
6.2 Rutas públicas y de invitados 29
6.3 Rutas de administración (role:admin) 29
6.4 Rutas compartidas (role:admin|empleado, prefijo admin) 30
6.5 Endpoints AJAX (JSON) 31
6.6 Generación y entrega de PDF 31
6.7 Integraciones y dependencias externas 31

7. Configuración, Variables de Entorno y Seguridad 33
7.1 Variables de entorno 33
7.2 Archivos de configuración propios 34
7.3 Controles de seguridad implementados 34
7.4 Hallazgos de la auditoría y recomendaciones 35
8. Guía de Despliegue (Deployment) y Entorno Local 36
8.1 Requisitos 36
8.2 Entorno local con Laragon 36
8.3 Despliegue en producción 37
8.4 Docker 37
8.5 Verificación posterior al despliegue 37
8.6 Problemas frecuentes 38

1. Resumen Ejecutivo y Propósito del Proyecto

1.1 Qué hace la aplicación

El **Sistema de Especies Municipales** es una aplicación web interna de la Unidad de Tesorería de la Alcaldía Municipal de Santa Ana Norte (El Salvador). Reúne dos subsistemas que comparten autenticación, usuarios y plantilla visual:

- **Especies Municipales (módulo principal).** Controla el ciclo de vida completo de las especies municipales valoradas y no valoradas (Fondo de Vialidad, tiquetes de mercado y de báscula, cartas de venta, fórmulas 1-ISAM, títulos a perpetuidad, guías de conducción de ganado, tarjetas, carnets, etc.): compra al Ministerio de Hacienda (M.H.), ingreso a una bodega central, traslado a los cinco distritos (D01 Metapán, D02 Masahuat, D03 Santa Rosa Guachipilín, D04 Texistepeque y D05 Santa Ana Norte), realización (venta o entrega), anulación de documentos y generación de los formularios oficiales de Tesorería en PDF.
- **Renta – Constancias de Retención.** Portal público, sin inicio de sesión, donde un contribuyente o empleado busca su NIT/DUI dentro de un archivo Excel anual y descarga su constancia de retención de renta en PDF. Cada búsqueda queda registrada para un reporte de auditoría de consultas.

1.2 Problema que resuelve

Antes del sistema, cada distrito llevaba su **Libro de Especies Municipales** y sus reportes mensuales («REALIZADAS» y «SALDOS») en hojas de Excel independientes. Durante la migración de los datos de noviembre de 2025 a junio de 2026 se documentaron errores típicos de ese esquema manual: series escritas incorrectamente, totales generales que no incluían filas nuevas, saldos anteriores que no coincidían con el cierre del mes previo, nulas contadas como vendidas y redondeos inconsistentes entre celdas.

El sistema sustituye ese control por un registro único y validado: cada documento se identifica por **tipo + serie + número**, cada movimiento se valida contra la historia completa del lote, y los reportes se calculan a partir de los movimientos, por lo que no pueden descuadrarse entre sí.

1.3 Propuesta de valor técnica

- **Inventario derivado, no almacenado.** Las existencias nunca se guardan en una columna; se calculan con aritmética de intervalos a partir de compras, traslados, realizaciones y nulas (`App\Services\Inventario`). Esto elimina la desincronización entre saldo y movimientos.
- **Validación temporal de cada movimiento.** Al registrar, editar o eliminar cualquier movimiento, el sistema «reproduce» cronológicamente la historia del lote afectado y rechaza el cambio si algún poseedor (bodega o distrito) hubiera entregado números que no tenía o recibido números que ya tenía (`Inventario::verificar`).
- **Concurrencia segura.** Las escrituras se serializan por tipo de especie con bloqueos de fila (`SELECT … FOR UPDATE`) dentro de transacciones; si la verificación posterior falla, la transacción se revierte.
- **Reportes oficiales parametrizables.** Los PDF reproducen los formatos de Tesorería (TESO-008-FORM, TESO-002-CONT, TESO-004-CONT, TESO-010-FORM). Títulos, descripciones, costos, precios de venta, orden de filas y filas de nulas se configuran por distrito desde la interfaz, sin modificar código.

- **Carga inicial reproducible.** Los datos históricos se importan desde un CSV normalizado mediante un seeder que vuelve a pasar todo por la bodega y verifica el inventario completo al final.

1.4 Stack tecnológico

Capa	Tecnología	Versión	Uso en el proyecto
Lenguaje (servidor)	PHP	8.4.12 (requiere ^8.3)	Lógica de negocio, controladores, servicios.
Framework	Laravel	13.8.0	Ruteo, ORM Eloquent, validación, sesiones, caché, migraciones, seeders.
Base de datos	MariaDB (compatible MySQL)	12.1.2	Motor InnoDB, conexión `mysql` de Laravel.
Interfaz de administración	jeroennoten/laravel-adminlte (AdminLTE 3, Bootstrap 4, jQuery, Font Awesome)	3.16.0	Plantilla, menú lateral con filtros por permiso, componentes visuales.
Roles y permisos	spatie/laravel-permission	7.4.1	Roles `admin`, `empleado`, `visitante`; middleware `role`.
Generación de PDF	carlos-meneses/laravel-mpdf (mPDF)	2.1.13 (mPDF 8.3.1)	Todos los reportes PDF y constancias de retención.
Lectura de Excel	phpoffice/phpspreadsheet	5.7.0	Búsqueda de NIT/DUI en los archivos de retención.
Alertas en navegador	SweetAlert2 (CDN jsDelivr)	11	Confirmaciones de eliminación y avisos.
Activos front-end (no usados por la app)	Vite + Tailwind CSS + laravel-vite-plugin	7 / 4 / 2	Solo la página `welcome` de Laravel; la interfaz real usa AdminLTE.
Pruebas y calidad	PHPUnit, Laravel Pint, Collision, Pail, Faker, Mockery	12.5 / 1.27 / 8.6 / 1.2 / 1.23 / 1.6	Dependencias de desarrollo.
Entorno local	Laragon (Apache/Nginx + MariaDB + PHP + Node 22)	—	Host virtual `proyectoserviciosocial.test`.

2. Arquitectura del Sistema y Patrones de Diseño

2.1 Arquitectura general

La aplicación es un **monolito MVC renderizado en el servidor** (Laravel + Blade). No expone una API pública ni utiliza un front-end SPA: cada pantalla es una vista Blade y la interactividad se resuelve con jQuery y unos pocos endpoints AJAX internos que devuelven JSON para llenar selectores dependientes.

El código de negocio de especies está organizado como un **módulo** dentro del monolito (`app/Http/Controllers/Especies`, `app/Services`, vistas `frontend/admin/especies`) y comparte con el resto de la aplicación la autenticación, la sesión y la base de datos.

Capa	Componentes	Responsabilidad
Entrada HTTP	`public/index.php`, `bootstrap/app.php`, `routes/web.php`	Controlador frontal, registro de rutas y alias de middleware.
Middleware	`web` (sesión, CSRF, cookies), `auth`, `guest`, `role:*` (Spatie), `no-back` (PreventBackHistory), `throttle:retencion`	Autenticación, autorización por rol, cabeceras anti-caché y límite de peticiones.
Controladores	15 controladores en `app/Http/Controllers` (7 en `Especies`)	Validan la petición, orquestan transacciones y devuelven vistas, redirecciones, JSON o PDF.
Servicios de dominio	`Inventario`, `Tarifas`	Cálculo de existencias por intervalos, validación temporal, bloqueos; tarifas por distrito.
Modelos (ORM)	15 modelos Eloquent en `app/Models`	Acceso a datos, relaciones, conversiones de tipos.
Vistas	Blade en `resources/views` (pantallas y plantillas PDF)	Presentación HTML y HTML para mPDF.
Configuración	`config/especies.php`, `config/adminlte.php`, tablas `distrito_tipo_especie` y `distrito_denominacion`	Códigos de formulario, firmas, menú y plantillas de reporte por distrito.
Persistencia	MariaDB; caché, sesiones y colas en tablas de la misma base	Datos de negocio y de infraestructura.

Flujo típico de una petición que modifica inventario (por ejemplo, registrar una realización):

```
Navegador ──POST /admin/especies/realizaciones──▶ web + auth + role:admin|empleado + no-back
  └─▶ RealizacionController@store
        ├─ validate()                          (reglas de formulario)
        ├─ datos(): denominación del tipo, monto = cantidad × precio de venta (Tarifas)
        └─ DB::transaction
             ├─ Inventario::bloquear(tipo)     (SELECT … FOR UPDATE)
             ├─ validarRango()                 (existe en compras, no realizado, no anulado)
             ├─ Realizacion::create()
             └─ Inventario::asegurar(pares)    (re-verifica la historia; si falla → rollback)
  └─▶ redirect() con mensaje (patrón Post/Redirect/Get)
```

2.2 Patrones de diseño implementados

Patrón	Dónde se aplica	Descripción
Modelo-Vista-Controlador (MVC)	Toda la aplicación	Separación entre rutas/controladores, modelos Eloquent y vistas Blade.

Patrón	Dónde se aplica	Descripción
Front Controller	`public/index.php`, `bootstrap/app.php`	Un único punto de entrada que delega en el router de Laravel.
Pipeline / Chain of Responsibility	Middleware de Laravel	Cada petición atraviesa una cadena de filtros (sesión, CSRF, auth, rol, throttle, no-back).
Active Record	Modelos Eloquent	Cada modelo encapsula su tabla y sus relaciones (`belongsTo`, `hasMany`).
Service Layer / Domain Service	`App\Services\Inventario`, `App\Services\Tarifas`	Lógica de dominio sin estado (métodos estáticos) reutilizada por controladores, reportes y seeders.
Estado derivado de un registro de movimientos (similar a event sourcing)	`Inventario::distrito`, `Inventario::bodega`, `Inventario::verificar`	La existencia es una función pura de los movimientos; no se guarda un saldo que pueda desincronizarse.
Bloqueo pesimista	`Inventario::bloquear`	`lockForUpdate()` sobre las filas de `tipo_especies` (en orden de id para evitar interbloqueos) serializa las escrituras de un mismo tipo.
Escritura + verificación + reversión (Transaction Script con invariantes)	`Inventario::asegurar` dentro de `DB::transaction`	Se escribe el cambio, se verifica la invariante del inventario y, si falla, la excepción revierte la transacción.
Strategy por configuración (Configuration over code)	`distrito_tipo_especie`, `distrito_denominacion`, `config/especies.php`	La forma de cada reporte (títulos, filas, orden, nulas, precios) se decide por datos, no por condiciones en código.
Plantillas parciales (Composite / Template View)	`reportes/pdf/header.blade.php`, `firma.blade.php`	Encabezado y firma comunes con banderas (`$encabezadoCentrado`, `$lineasTitulo`, `$sinFechaGeneracion`).
Post/Redirect/Get	Todos los formularios	Tras un POST/PATCH/DELETE se redirige con mensajes de sesión para evitar reenvíos.
Token de recurso temporal	`RetencionController`, `ConsultaController`	El PDF se guarda en caché bajo un UUID v4 durante 30 minutos y se sirve por `GET …/pdf/{token}`.
Control de acceso basado en roles (RBAC)	Spatie Permission + Gates	Middleware `role:` en rutas y Gates (`admin-access`, `shared-access`, `auth-access`) para el menú.
Rate Limiter	`AppServiceProvider`, `LoginController`	Límites por IP y por usuario contra fuerza bruta y enumeración de NIT/DUI.
Pipeline de importación (ETL)	`DatosInicialesEspeciesSeeder`	Lee el CSV, valida cada fila, recrea los movimientos por la bodega y verifica el inventario completo.

> *No se usan los patrones Repository ni Factory de dominio: los controladores consultan Eloquent directamente y la única fábrica es `UserFactory` (plantilla de Laravel). Para el tamaño actual del sistema esto es adecuado; ver recomendaciones en la sección 7.4.*

3. Estructura de Directorios y Organización del Código

Árbol comentado del repositorio (se omiten `vendor/`, `node_modules/` y archivos de caché):

```
ProyectoServicioSocial/
├── app/
│   ├── Http/
│   │   ├── Controllers/
│   │   │   ├── Auth/LoginController.php         Login por usuario y límite de intentos
│   │   │   ├── AnioController.php               Años de retención y su archivo Excel
│   │   │   ├── ConsultaController.php           Reporte PDF de consultas por periodo
│   │   │   ├── DashboardController.php          Redirección post-login y paneles por rol
│   │   │   ├── RetencionController.php          Búsqueda pública NIT/DUI y constancia PDF
│   │   │   ├── RolController.php                Cambio de rol de usuarios (solo admin)
│   │   │   ├── UsuarioController.php            CRUD de usuarios (solo admin)
│   │   │   └── Especies/
│   │   │       ├── ConfiguracionController.php  Tipos de especie y denominaciones (+ AJAX)
│   │   │       ├── DistritoConfigController.php Títulos y tarifas por distrito
│   │   │       ├── CompraController.php         Compras al M.H., lotes y rangos
│   │   │       ├── BodegaController.php         Traslados, detalles, stock de bodega (+ AJAX)
│   │   │       ├── DistritoController.php       Anulaciones (nulas) y stock por distrito (+ AJAX)
│   │   │       ├── RealizacionController.php    Realizaciones (ventas/entregas) (+ AJAX)
│   │   │       └── ReporteController.php        Los 8 reportes PDF del módulo
│   │   └── Middleware/PreventBackHistory.php    Cabeceras no-cache para páginas autenticadas
│   ├── Models/                                  15 modelos Eloquent (ver sección 5)
│   ├── Providers/AppServiceProvider.php         Gates, paginación, límite de búsquedas
│   └── Services/
│       ├── Inventario.php                       Motor de existencias y validación temporal
│       └── Tarifas.php                          Costo y precio de venta por distrito
├── bootstrap/app.php                            Rutas y alias de middleware
├── config/
│   ├── adminlte.php                             Título, logo, menú y plugins
│   ├── especies.php                             Códigos de formulario y firma
│   ├── permission.php                           Configuración de Spatie
│   └── app.php, database.php, session.php …     Configuración estándar de Laravel
├── database/
│   ├── data/
│   │   ├── especies_nov2025_jun2026.csv         Movimientos nov-2025 a jun-2026
│   │   └── *.sql                                Volcados de base de datos (ver riesgos en 7.4)
│   ├── factories/UserFactory.php                Fábrica de usuarios (plantilla)
│   ├── migrations/                              27 migraciones (ver 5.6)
│   └── seeders/                                 Roles, catálogo y datos iniciales
├── lang/                                        Traducciones
├── public/
│   ├── index.php                                Controlador frontal
│   ├── constancias/*.blade.php                  Plantillas de constancias (mPDF)
│   ├── excel/*.xlsx                             Excel de retención (datos personales)
│   ├── images/                                  Logos, firma y sello usados en PDF y menú
│   ├── reportes/Reporte_consultas.blade.php     Plantilla del reporte de consultas
│   └── vendor/                                  AdminLTE, Bootstrap, jQuery, FA
├── resources/views/
│   ├── errors/                                  Páginas 403, 404, 419 y 500 personalizadas
│   ├── frontend/layouts/                        admin.blade.php y empleado.blade.php (AdminLTE)
│   ├── frontend/auth/login.blade.php            Formulario de inicio de sesión
│   ├── frontend/retencion/index.blade.php       Portal público de constancias
│   ├── frontend/admin/                          Usuarios, roles, años, consultas, dashboard
│   └── frontend/admin/especies/
│       ├── configuracion/                       tipos, denominaciones, distritos
│       ├── compras/                             historial, crear, show, lotes/crear
│       ├── bodega/                              traslados y stock
│       ├── distritos/                           anulaciones y stock
│       ├── realizaciones/                       historial, crear, editar
│       └── reportes/                            filtros y pdf/ (plantillas mPDF)
├── routes/web.php                               Todas las rutas web
├── storage/                                     Logs, caché de vistas, temporales
├── tests/                                       Pruebas de ejemplo de Laravel (Feature/Unit)
├── _referencia_actualizada/                     Libros Excel de Tesorería (fuera de Git)
```

```
├── composer.json / composer.lock                Dependencias PHP
├── package.json / vite.config.js                Front-end (solo página welcome)
└── .env / .env.example                          Variables de entorno
```

> *Existen vistas antiguas que ninguna ruta utiliza: `especies/realizaciones/registrar.blade.php`, `especies/compras/registrar.blade.php`, `especies/bodega/traslado_registrar.blade.php`, `especies/bodega/traslado_historial.blade.php` y `especies/distritos/anulacion_registrar.blade.php`. Pueden eliminarse sin efecto funcional.*

4. Módulos y Lógica de Negocio

4.1 Mapa de módulos

Módulo	Componentes principales	Acceso	Función
Autenticación	`Auth\LoginController`, `frontend/auth/login`	Invitados	Inicio de sesión por nombre de usuario, límite de intentos, redirección según rol, cierre de sesión.
Paneles	`DashboardController`	Autenticados	`/home` redirige: admin y empleado → `admin.dashboard`; visitante → `empleado.dashboard`.
Usuarios y roles	`UsuarioController`, `RolController`	admin	Alta, edición y baja de usuarios (rol inicial `empleado`); cambio de rol con `syncRoles`.
Años de retención	`AnioController`	admin, empleado	Periodos fiscales y archivo Excel asociado (se elige entre los `.xlsx/.xls` de `public/excel`).
Constancias de retención	`RetencionController`, `public/constancias`	Público (con límite por IP)	Búsqueda de NIT/DUI y emisión de la constancia PDF.
Reporte de consultas	`ConsultaController`	admin, empleado	PDF con las búsquedas de constancias agrupadas por NIT/DUI en un periodo.
Especies – Configuración	`Especies\ConfiguracionController`, `Especies\DistritoConfigController`	admin, empleado	Tipos de especie, denominaciones, datos de distrito, títulos de reportes y tarifas por distrito.
Especies – Compras	`Especies\CompraController`	admin, empleado	Compras al M.H. (factura) con sus lotes y rangos de numeración: ingreso a bodega.
Especies – Bodega	`Especies\BodegaController`	admin, empleado	Traslados (bodega→distrito, devoluciones, entre distritos), detalles y existencias de bodega.
Especies – Distritos	`Especies\DistritoController`	admin, empleado	Anulaciones (nulas) con edición y existencias por distrito.
Especies – Realizaciones	`Especies\RealizacionController`	admin, empleado	Registro, edición y eliminación de rangos realizados (vendidos/entregados).
Especies – Reportes	`Especies\ReporteController`, vistas `reportes/pdf`	admin, empleado	Ocho reportes PDF oficiales y de control.

4.2 Modelo de dominio de especies

Un documento (tiquete, carta, fórmula, etc.) se identifica de forma única por **tipo de especie + serie + número**. La misma numeración se repite entre series distintas, por lo que toda comparación de rangos se acota por tipo y serie. Las cantidades se manejan siempre como **rangos cerrados** `[inicio, fin]`, nunca documento por documento.

Entidad	Significado de negocio
Tipo de especie (`tipo_especies`)	Clase de documento. `unidades = 1` indica que el tipo se lleva en unidades (fórmulas, cartas, guías, tarjetas de control): en el libro el «monto» es la cantidad y no se valúa.

Entidad	Significado de negocio
Denominación (`denominaciones`)	Valor facial o de libro dentro de un tipo (por ejemplo, Fondo Vialidad $0.57, $1.14, $3.43). Guarda descripción, precio de costo (4 decimales) y precio de venta general (null = no se vende).
Compra (`compras`)	Factura de compra al Ministerio de Hacienda. Su `monto_total` se recalcula como Σ(cantidad del lote × valor de la denominación).
Lote y rangos (`lotes`, `lote_rangos`)	Documentos de un tipo, denominación y serie dentro de una compra; uno o varios rangos de numeración que no pueden solaparse con otro lote del mismo tipo y serie.
Traslado y detalles (`traslados`, `traslado_detalles`)	Movimiento fechado entre poseedores. `tipo`: `bodega_distrito` (envío), `distrito_bodega` (devolución) o `distrito_distrito`. Cada detalle mueve un rango de un lote.
Nula (`nulas`)	Rango anulado (dañado, mal impreso, etc.) en un distrito. Se vincula al detalle de traslado por el que llegó el documento.
Realización (`realizaciones`)	Rango vendido o entregado en un distrito, con fecha, cantidad y monto cobrado. No referencia el lote: se ubica por tipo + serie + número.
Distrito y su configuración	`distritos` (código D01–D05, nombre oficial, lugar de firma, orden del saldo), `distrito_tipo_especie` (títulos y orden por tipo) y `distrito_denominacion` (tarifas: costo, venta, rótulos, orden y filas de nulas).

Existen dos clases de **poseedores**: la bodega central y cada distrito. El efecto de cada movimiento sobre ellos es:

Movimiento	Bodega	Distrito origen	Distrito destino
Compra (lote con rangos)	+ rangos del lote	—	—
Traslado `bodega_distrito`	−	—	+
Devolución `distrito_bodega`	+	−	—
Traslado `distrito_distrito`	—	−	+
Realización	—	− (distrito que realiza)	—
Nula	—	− (distrito que anula)	—

> *Regla de negocio: todo documento entra al sistema por la bodega (compra) y luego se distribuye a los distritos por traslado. Incluso las existencias iniciales al 31/10/2025 se cargaron como una compra «SALDO-INICIAL» trasladada el mismo día a cada distrito.*

4.3 Motor de inventario (App\Services\Inventario)

Las existencias no se almacenan: se derivan de los movimientos cada vez que se consultan. El servicio expone métodos estáticos sin estado, usados por controladores, reportes y el seeder de carga inicial.

4.3.1 Aritmética de intervalos: neto()

Es el núcleo del motor. Recibe una lista de rangos que suman (`$mas`), otra que restan (`$menos`) y un umbral (`$minimo`), y devuelve los intervalos disjuntos, ordenados y fusionados donde el conteo neto es mayor o igual al umbral. Implementa un **arreglo de diferencias con barrido (sweep line)**:

1. Por cada rango `[i, f]` de `$mas` suma 1 en la posición `i` y resta 1 en `f + 1`; para `$menos` hace lo inverso.
2. Ordena las posiciones con `ksort` y recorre acumulando el conteo.
3. Abre un intervalo cuando el conteo cruza el umbral hacia arriba y lo cierra (en `pos − 1`) cuando cae por debajo.

La complejidad es O(k log k), donde k es el número de rangos, independiente de la cantidad de documentos (un rango de 300 000 números cuesta lo mismo que uno de 10). Ejemplo: `neto([[1,100]], [[20,30],[50,50]])` devuelve `[[1,19],[31,49],[51,100]]`.

```
public static function neto(array $mas, array $menos = [], int $minimo = 1): array
{
    $delta = [];
    foreach ($mas as [$i, $f]) {
        $delta[$i]     = ($delta[$i] ?? 0) + 1;
        $delta[$f + 1] = ($delta[$f + 1] ?? 0) - 1;
    }
    foreach ($menos as [$i, $f]) {
        $delta[$i]     = ($delta[$i] ?? 0) - 1;
        $delta[$f + 1] = ($delta[$f + 1] ?? 0) + 1;
    }
    ksort($delta);
    $res = []; $conteo = 0; $desde = null;
    foreach ($delta as $pos => $d) {
        $antes = $conteo; $conteo += $d;
        if ($antes < $minimo && $conteo >= $minimo) {
            $desde = $pos;
        } elseif ($antes >= $minimo && $conteo < $minimo) {
            $res[] = [$desde, $pos - 1];
        }
    }
    return $res;
}
```

Operaciones derivadas:

- `interseccion(a, b)` = `neto(neto(a) ∪ neto(b), [], 2)`: normaliza cada lado (para que un solapamiento interno no cuente doble) y conserva las posiciones presentes en ambos.
- `contiene(intervalos, i, f)` es verdadero si `neto([[i, f]], intervalos)` queda vacío.
- `total()` suma `fin − inicio + 1`; `texto()` formatea «1,001–1,050, 1,060» para los mensajes de error.
- `serie()` normaliza la serie a mayúsculas y convierte la cadena vacía en `null`.

4.3.2 Existencias de un distrito: distrito()

`distrito(distritoId, tipoId?, hasta?, sinRealizacion?)` devuelve, por cada lote que el distrito haya recibido alguna vez (aunque hoy tenga cero), los intervalos vivos y los contadores `recibido`, `salido`, `anulado`, `realizado`, `cantidad`, `valor` y `monto`. Realiza cuatro consultas agrupadas (entradas por traslado, salidas por traslado, nulas y realizaciones, todas filtradas opcionalmente hasta una fecha) y luego calcula por lote:

```
intervalos = neto(recibido, salido ∪ anulado ∪ (realizaciones[tipo|serie] ∩ recibido))
```

Como las realizaciones no guardan el lote, se toman las de su tipo y serie y se intersecan con lo que el distrito recibió de ese lote. El parámetro `sinRealizacion` excluye una realización concreta (se usa al editarla para mostrar su rango como disponible). El resultado se ordena por valor, serie e id de lote.

4.3.3 Existencias de bodega: bodega()

`bodega(tipoId?, hasta?)` parte de los rangos de cada lote comprado hasta la fecha, suma las devoluciones (`distrito_bodega`) y resta los envíos (`bodega_distrito`): `neto(rangos ∪ devueltos, enviados)`.

4.3.4 Validación temporal: verificar()

`verificar(distritoId|null, loteId)` reproduce cronológicamente la historia de un lote en un poseedor (bodega si es `null`) y devuelve el primer error en lenguaje natural, o `null` si la historia es válida. Detecta dos tipos de inconsistencia: entregar números que el poseedor no tenía en esa fecha y recibir números que ya tenía.

```
eventos = (fecha, +1, rango)  por compra, devoluciones o traslados recibidos
        + (fecha, −1, rango)  por envíos, nulas y realizaciones (n rangos del lote)
tiene = []
para cada fecha (ascendente), con los eventos del día como un solo bloque:
    mas   = tiene ∪ entradas del día
    menos = salidas del día
    si neto(mas, menos, 2) ≠ ∅  →  error "… ya tenía los números … cuando llegó el traslado #N"
    si neto(menos, mas)    ≠ ∅  →  error "… no tenía disponibles los números … para la realización"
    tiene = neto(mas, menos)
retornar null
```

Las fechas no tienen hora, así que cada día se evalúa como un bloque sin suponer un orden interno: un rango recibido y realizado el mismo día es válido.

4.3.5 Concurrencia y transacciones: bloquear() y asegurar()

Toda operación que modifica inventario sigue el mismo esquema dentro de `DB::transaction`:

1. `Inventario::bloquear(...tipos)` ejecuta `SELECT … FOR UPDATE` sobre las filas de `tipo_especies` afectadas, en orden de id (orden consistente para evitar interbloqueos). Así se serializan las escrituras concurrentes de un mismo tipo.
2. Se aplican las validaciones propias del formulario y se escribe el cambio.
3. `Inventario::asegurar(pares)` llama a `verificar()` para cada par `[poseedor, lote]` afectado (sin repetir) y, ante el primer error, lanza `ValidationException`. La excepción revierte la transacción y el mensaje se muestra en el formulario.

Operación	Pares verificados
Agregar o quitar un detalle de traslado	(origen, lote) y (destino, lote); `null` representa la bodega.
Editar fecha o distritos de un traslado	Pares antes del cambio + pares después del cambio.
Eliminar un traslado	Todos los pares del traslado (bloqueado si tiene nulas).
Editar fecha de una compra	(bodega, cada lote de la compra).
Registrar o editar realización	(distrito, cada lote que contiene parte del rango), antes y después.
Registrar o editar anulación	(distrito, lote) y, al editar, el par anterior.
Eliminar realización o anulación	Sin verificación: solo devuelve números a un poseedor que ya los tenía.

4.3.6 Otras funciones

- `filasSaldo(distrito, tipo, hasta, denominación?)`: una fila por rango vivo, como la lista el libro. Se ordena por valor (si el distrito tiene `saldo_por_valor`), id de lote e inicio; Texistepeque (D04) usa el orden de ingreso.

- `lotesDelRango(tipo, serie, inicio, fin)`: lotes del tipo y serie que contienen parte del rango; se usa para saber qué historias toca una realización.

> *Consideración de rendimiento: los cálculos son en línea y `verificar()` recorre toda la historia del lote. Con los volúmenes actuales (≈2 000 movimientos) la respuesta es inmediata; si el histórico crece durante años, conviene evaluar cierres mensuales o un índice adicional en `realizaciones(distrito_id, tipo_especie_id, serie)` y `nulas(distrito_id, fecha)`.*

4.4 Flujos de negocio

4.4.1 Compra al M.H. e ingreso a bodega

1. Se crea la compra con número de factura único, fecha (no futura) y observaciones; `monto_total` inicia en 0 y se guarda el usuario.
2. Se agregan lotes: tipo, denominación (debe pertenecer al tipo), serie opcional y uno o más rangos. Se valida inicio ≤ fin, que los rangos del mismo envío no se solapen y, dentro de una transacción con bloqueo, que no se solapen con ningún lote existente del mismo tipo y serie.
3. Se recalcula `monto_total`. Un lote o una compra solo se pueden eliminar si no tienen traslados; cambiar la fecha de la compra re-verifica la bodega (no puede quedar un envío anterior a la compra).

4.4.2 Traslados

Se registran en dos pasos: encabezado (tipo, origen y/o destino, fecha, observaciones) y detalles (lote + rango). El tipo no puede cambiarse después de creado. Cada detalle debe estar contenido en los rangos del lote y la verificación temporal garantiza que el origen tenía esos números en la fecha y que el destino no los tenía ya. En el historial, un traslado sin detalles muestra el botón «Agregar detalles».

4.4.3 Realizaciones

1. El formulario consulta por AJAX (`ajax/realizacion-info`) lo que el distrito tiene del tipo seleccionado, agrupado por denominación y serie, con sus rangos disponibles y el precio de venta del distrito.
2. Al guardar: `monto_cobrado = round(cantidad × precio de venta del distrito, 2)` (0 si la especie no se vende en ese distrito).
3. `validarRango()` exige que el rango exista completo en compras de esa denominación, tipo y serie; que ninguna parte esté ya realizada (en cualquier distrito) ni anulada.
4. `Inventario::asegurar()` confirma que el distrito tenía esos números en la fecha indicada.

4.4.4 Anulaciones

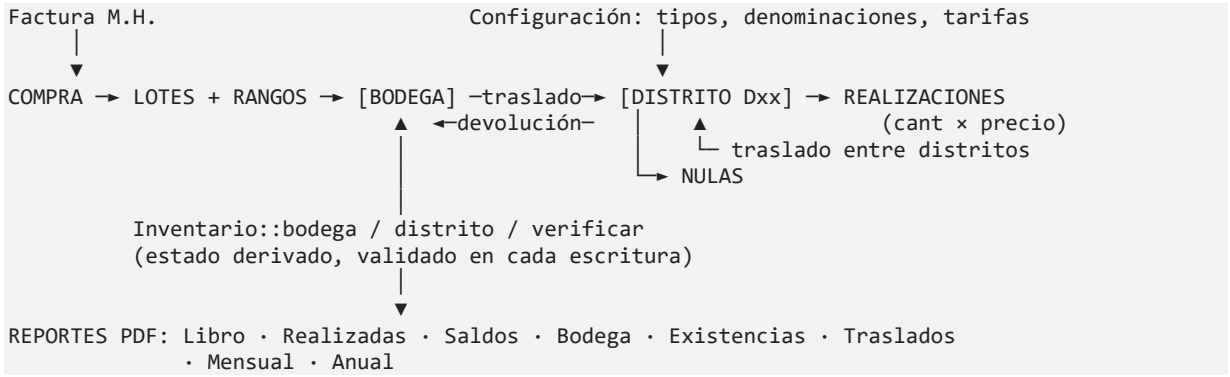
Se elige distrito y tipo; el AJAX `ajax/detalles-disponibles` devuelve los detalles de traslado recibidos por el distrito con lo que de cada uno sigue disponible. Reglas: el detalle pertenece al distrito, el rango está dentro del detalle, un número se anula una sola vez por lote, no se puede anular lo ya realizado y el distrito debía tener los números en la fecha. La edición reutiliza el mismo método (`guardarAnulacion`) e incluye en la verificación el lote y distrito anteriores; el rango de la propia nula se muestra como disponible mientras se edita.

4.4.5 Tarifas por distrito y recálculo del monto cobrado

`App\Services\Tarifas::de(distrito, denominación)` resuelve descripción, costo y precio de venta con esta precedencia: tarifa del distrito (`distrito_denominacion`) y, si no existe, los valores generales de la denominación. En una tarifa, `precio_venta = null` es una decisión explícita: esa especie no se vende en el distrito.

Al guardar una tarifa desde Configuración de Distritos, si el usuario cambia el precio de venta, la interfaz ofrece «Guardar y recalcular» con una fecha `aplicar_desde`: se recalcula `monto_cobrado = round(cantidad × nuevo precio, 2)` de todas las realizaciones de ese distrito y denominación con fecha mayor o igual a la indicada.

4.4.6 Flujo de datos de extremo a extremo



4.5 Reportes PDF

Todos los reportes son peticiones GET con el parámetro `generar`; si falta, se muestra el formulario de filtros. Se generan con mPDF (márgenes de 15 mm) y se envían con `stream()` para visualizarse en el navegador. Los encabezados y firmas son parciales Blade comunes (`reportes/pdf/header.blade.php` y `firma.blade.php`) parametrizados por banderas. Los códigos, versiones, vigencia (11/11/2025) y la firma de la tesorera provienen de `config/especies.php`.

Reporte	Filtros	Formulario	Fuente de datos
Libro de Especies Municipales	Distrito, tipo, mes, año, denominación (opcional)	TESO-008-FORM v001	`datosLibro()` + `Inventario::filasSaldo()`
Especies Realizadas (resumen y detallado)	Distrito, mes, año, tipo (opcional), casilla «detallado»	TESO-002-CONT v001	`datosRealizaciones()` / `datosRealizacionesDetalle()`
Control de Saldos	Distrito, mes, año	TESO-004-CONT v000	`datosSaldos()` + `Inventario::distrito()`
Especies en Bodega	Tipo, fecha desde, fecha hasta	TESO-010-FORM v000	Lotes comprados en el periodo, enviado y devuelto hasta la fecha
Existencias por Distrito	Distrito, tipo, fecha de corte	TESO-000-FORM	`Inventario::distrito()` a la fecha de corte
Historial de Traslados	Distrito, tipo, desde, hasta	TESO-000-FORM	Traslados y detalles del periodo
Reporte Mensual	Distrito o «todos», mes, año	TESO-000-FORM	Saldos y movimientos del mes por tipo

Reporte	Filtros	Formulario	Fuente de datos
Consolidado Anual	Distrito, tipo, año	TESO-000-FORM	Movimientos agregados por mes

4.5.1 Libro de Especies Municipales

Reproduce el orden de los libros de Tesorería: SALDO ANTERIOR (filas de saldo al último día del mes anterior), POR COMPRA AL M.H. FACTURA N° (traslados desde bodega agrupados por traslado y factura), POR TRASLADOS (recibidos de otro distrito), POR REALIZACIONES EN EL MES, NULAS, POR TRASLADOS (enviados) y POR SALDO A NUEVA CUENTA (saldo al cierre). Con `nulas_en_realizaciones` el total de realizaciones incluye las nulas. Los tipos en unidades muestran cantidades en lugar de montos. El filtro de denominación acota todas las secciones para que el libro cuadre.

4.5.2 Especies Realizadas

Genera una fila por tarifa del distrito que tuvo existencia al inicio o al final del mes, o que tuvo movimiento (aunque quede en cero). Para cada fila: cantidad realizada (más las nulas si el distrito no lleva fila propia de nulas), venta = suma de `monto_cobrado` y descargo = cantidad × precio de costo **truncado al centavo**. Si la tarifa define `nulas_realizaciones`, las nulas van en una fila aparte con su propio orden. Los grupos se ordenan por el orden del tipo en el distrito y las filas por `orden_realizaciones`. La versión detallada lista cada rango (denominación, serie, del, al, cantidad, monto cobrado, descargo) y una subsección de NULAS, sin encabezado ni firma.

4.5.3 Control de Saldos

Construye una plantilla por tipo a partir de las tarifas del distrito (más las denominaciones con existencia que no tengan tarifa, al final con orden 900). Cada fila muestra la existencia al cierre del mes, el precio de venta, la venta potencial y las existencias a costo, ambas truncadas al centavo. Las filas de nulas configuradas (`descripcion_nulas`) salen siempre en cero, como en las hojas originales. Los montos en cero se muestran como «$ -».

4.5.4 Truncado monetario

```
private function truncar(float $v): float
{
    return floor((float) sprintf('%.15g', $v * 100)) / 100;   // 22.875 → 22.87
}
```

El paso por `sprintf('%.15g')` elimina el ruido binario de punto flotante antes de truncar (por ejemplo, 29 758 × 0.015 = 446.37 y no 446.36). Los totales suman los valores ya truncados de cada fila, igual que las hojas de Tesorería.

4.6 Constancias de retención y reporte de consultas

1. El usuario elige un año y escribe su NIT/DUI en `/retencion`. La ruta `POST retencion/buscar` está limitada a 20 búsquedas por minuto por IP para invitados (sin límite para personal autenticado).
2. Se abre con PhpSpreadsheet el Excel del año (`public/excel/<archivo>`) y se recorre la columna C de las hojas «COD 01-60-80-81» y «COD 11-27-70-84» comparando el NIT/DUI normalizado (sin espacios, guiones ni puntos, en mayúsculas).
3. Con cada coincidencia se renderiza la plantilla Blade correspondiente de `public/constancias` (columnas B–L: nombre, NIT, código, monto devengado, impuesto retenido, aguinaldo exento y

gravado, ISSS, AFP, IPSFA, INPEP). Si hay dos constancias se combinan en un solo PDF con `<pagebreak />`.

4. Toda búsqueda se registra en `consultas` (NIT/DUI, nombre encontrado, fecha y hora), encuentre o no resultados.
5. El PDF se guarda en caché codificado en base64 bajo la clave `retencion_pdf_<uuid>` durante 30 minutos; la página lo muestra en un iframe desde `GET retencion/pdf/{token}`.

El reporte de consultas (`admin/consultas`) agrupa las búsquedas de un periodo por NIT/DUI con su conteo y usa el mismo mecanismo de token en caché (`reporte_consultas_pdf_<uuid>`).

4.7 Usuarios, roles y paneles

Rol	Acceso
admin	Todo el sistema, incluidos usuarios y roles.
empleado	Panel, años, especies (configuración, bodega, distritos, realizaciones, reportes) y consultas.
visitante	Solo `empleado/dashboard`.

Los usuarios se crean con rol `empleado` y contraseña de mínimo 6 caracteres (cifrada por el cast `hashed`). Cuando un administrador cambia su propio rol, la sesión se cierra para que vuelva a iniciar con los permisos nuevos. El menú lateral de AdminLTE se filtra con los Gates `admin-access`, `shared-access` y `auth-access`.

4.8 Carga inicial de datos (ETL)

Los movimientos de noviembre de 2025 a junio de 2026 se obtuvieron de los libros Excel de Tesorería (`_referencia_actualizada`) y se normalizaron en `database/data/especies_nov2025_jun2026.csv` con las columnas `fecha, movimiento, distrito, destino, tipo_especie, valor, serie, del, al, cantidad, factura, nota`.

Movimiento	Filas	Tratamiento en DatosInicialesEspeciesSeeder
SALDO_INICIAL	84	Compra «SALDO-INICIAL» (existencias al 31/10/2025) trasladada el mismo día a cada distrito.
COMPRA	33	Una compra por factura con un lote por fila, trasladada el mismo día al distrito que la recibió.
TRASLADO	3	Traslado `distrito_distrito`; el rango se parte según el detalle por el que llegó al distrito de origen.
REALIZACION	1 028	Realización con `monto_cobrado` = cantidad × precio de venta del distrito.
NULA	769	Nula vinculada al detalle de traslado por el que llegó el rango.

El seeder solo se ejecuta con las tablas de movimientos vacías, carga todo en una única transacción y al final llama a `Inventario::verificar()` para la bodega y cada par distrito–lote; cualquier inconsistencia aborta la carga completa.

5. Base de Datos y Modelo de Datos

5.1 Motor y características generales

Aspecto	Valor
Motor	MariaDB 12.1.2 (driver `mysql` de Laravel; compatible con MySQL 8).
Base de datos	`db_proyecto_renta` (nombre heredado del módulo de renta).
Motor de almacenamiento	InnoDB en todas las tablas (transacciones, claves foráneas y bloqueos de fila).
Juego de caracteres	`utf8mb4` con colación `utf8mb4_unicode_ci`.
Número de tablas	27: 15 de negocio, 5 de Spatie Permission y 7 de infraestructura de Laravel (`sessions`, `cache`, `cache_locks`, `jobs`, `job_batches`, `failed_jobs`, `migrations`).
Tipos monetarios	`DECIMAL(12,2)` para montos, `DECIMAL(10,2)` para valores y `DECIMAL(10,4)` para costos y precios (hay costos como 0.0105).
Numeración de documentos	`INT UNSIGNED` (validado hasta 999 999 999).

5.2 Diagrama entidad-relación

Notación: `A ──< B` indica que un registro de A tiene muchos de B (B contiene la clave foránea).

```
tipo_especies ──< denominaciones
      │                  │
      └──────< lotes >───┘          compras ──< lotes ──< lote_rangos
                  │
                  └──< traslado_detalles >── traslados >── distritos   (distrito_id = destino)
                                                  └──────>── distritos   (origen_distrito_id)
                              │
                              └──< nulas >── distritos

realizaciones >── tipo_especies, denominaciones, distritos          (sin clave foránea a lotes)
distrito_tipo_especie >── distritos, tipo_especies                  (único por par)
distrito_denominacion >── distritos, denominaciones                 (único por par)
users ──< compras, traslados, nulas, realizaciones                  (user_id / usuario_id)
users >──< roles                                                    (Spatie: model_has_roles)
anios, consultas                                                    (independientes)
```

> *`realizaciones` no tiene clave foránea a `lotes`: se relaciona con el inventario por tipo + serie + número, y el motor de intervalos resuelve a qué lote pertenece cada número.*

5.3 Relaciones Eloquent

Modelo	Relaciones
TipoEspecie	`hasMany` denominaciones
Denominacion	`belongsTo` tipoEspecie; `hasMany` lotes, realizaciones; accesor `etiqueta` (descripción o valor formateado)
Distrito	`hasMany` traslados, tiposConfig (DistritoTipoEspecie), tarifas (DistritoDenominacion); accesor `nombre_reporte`
DistritoTipoEspecie / DistritoDenominacion	`belongsTo` distrito y tipoEspecie / denominacion
Compra	`hasMany` lotes; `belongsTo` user
Lote	`belongsTo` compra, tipoEspecie, denominacion; `hasMany` rangos (LoteRango)

Modelo	Relaciones
Traslado	`belongsTo` distrito (destino), origenDistrito, usuario; `hasMany` detalles
TrasladoDetalle	`belongsTo` traslado, lote
Nula	`belongsTo` trasladoDetalle, distrito, usuario; accesor `cantidad`
Realizacion	`belongsTo` tipoEspecie, denominacion, distrito, usuario
User	Trait `HasRoles` de Spatie; cast `password => hashed`
Anio, Consulta	Sin relaciones

5.4 Diccionario de datos

Columnas obtenidas de `information_schema` de la base en funcionamiento. «FK/IDX» indica una columna con índice no único (normalmente la clave foránea).

Tabla tipo_especies

Catálogo de tipos de especie (17 registros).

Columna	Tipo	Nulo	Clave	Descripción
`id`	bigint(20) unsigned	No	PK	Identificador autoincremental.
`nombre`	varchar(100)	No		Nombre del tipo.
`descripcion`	text	Sí		Descripción opcional.
`unidades`	tinyint(1)	No		1 = se lleva en unidades (sin valuación en el libro).
`orden`	smallint(5) unsigned	No		Orden general (se asigna automáticamente al final).
`activo`	tinyint(1)	No		Disponible para nuevos registros.
`created_at`	timestamp	Sí		Marca de tiempo de Eloquent.
`updated_at`	timestamp	Sí		Marca de tiempo de Eloquent.

Tabla denominaciones

Valores de cada tipo con costo y precio de venta generales (27 registros).

Columna	Tipo	Nulo	Clave	Descripción
`id`	bigint(20) unsigned	No	PK	Identificador autoincremental.
`tipo_especie_id`	bigint(20) unsigned	No	FK/IDX	Tipo al que pertenece.
`descripcion`	varchar(150)	Sí		Rótulo usado en libros y reportes.
`valor`	decimal(10,2)	No		Valor facial o de libro.
`precio_costo`	decimal(10,4)	Sí		Precio de costo general (base del descargo).
`precio_venta`	decimal(10,4)	Sí		Precio de venta general; null = no se vende.

Columna	Tipo	Nulo	Clave	Descripción
activo	tinyint(1)	No		Disponible para nuevos registros.
created_at	timestamp	Sí		Marca de tiempo de Eloquent.
updated_at	timestamp	Sí		Marca de tiempo de Eloquent.

Tabla distritos

Distritos D01–D05.

Columna	Tipo	Nulo	Clave	Descripción
id	bigint(20) unsigned	No	PK	Identificador autoincremental.
nombre	varchar(100)	No		Nombre corto (ej. «Distrito Metapán»).
nombre_oficial	varchar(150)	Sí		Nombre para encabezados de reportes.
lugar_firma	varchar(150)	Sí		Lugar que aparece en el pie del libro.
saldo_por_valor	tinyint(1)	No		1 = saldo ordenado por valor; 0 = por orden de ingreso.
codigo	varchar(10)	No	UNI	Código D01–D05 (único).
activo	tinyint(1)	No		Aparece en los filtros de reportes.
created_at	timestamp	Sí		Marca de tiempo de Eloquent.
updated_at	timestamp	Sí		Marca de tiempo de Eloquent.

Tabla distrito_tipo_especie

Títulos, orden y forma de sumar nulas de cada tipo en cada distrito (37 registros).

Columna	Tipo	Nulo	Clave	Descripción
id	bigint(20) unsigned	No	PK	Identificador autoincremental.
distrito_id	bigint(20) unsigned	No	FK/IDX	Clave foránea.
tipo_especie_id	bigint(20) unsigned	No	FK/IDX	Clave foránea.
titulo_libro	varchar(150)	Sí		Título del tipo en el libro.
titulo_reporte	varchar(150)	Sí		Título del grupo en Control de Saldos.
titulo_realizaciones	varchar(150)	Sí		Título del grupo en Especies Realizadas.
orden	smallint(5) unsigned	No		Orden del tipo en los reportes del distrito.
nulas_en_realizaciones	tinyint(1)	No		El libro suma las nulas al total de realizaciones.
created_at	timestamp	Sí		Marca de tiempo de Eloquent.
updated_at	timestamp	Sí		Marca de tiempo de Eloquent.

Tabla distrito_denominacion

Tarifa de cada denominación en cada distrito y plantilla de filas de reportes (46 registros).

Columna	Tipo	Nulo	Clave	Descripción
id	bigint(20) unsigned	No	PK	Identificador autoincremental.
distrito_id	bigint(20) unsigned	No	FK/IDX	Clave foránea.
denominacion_id	bigint(20) unsigned	No	FK/IDX	Clave foránea.
descripcion	varchar(150)	Sí		Rótulo de la fila en Control de Saldos.
precio_costo	decimal(10,4)	Sí		Costo propio del distrito (prevalece sobre el general).
precio_venta	decimal(10,4)	Sí		Precio de venta en el distrito; null = no se vende.
orden	smallint(5) unsigned	No		Orden de la fila en Control de Saldos.
descripcion_nulas	varchar(150)	Sí		Rótulo de la fila de nulas en Control de Saldos.
orden_nulas	smallint(5) unsigned	Sí		Orden de la fila de nulas en saldos.
nulas_con_venta	tinyint(1)	No		La fila de nulas muestra venta en cero.
descripcion_realizaciones	varchar(150)	Sí		Rótulo de la fila en Especies Realizadas.
orden_realizaciones	smallint(5) unsigned	Sí		Orden de la fila en Especies Realizadas.
nulas_realizaciones	varchar(150)	Sí		Rótulo de la fila propia de nulas en Realizadas.
orden_nulas_realizaciones	smallint(5) unsigned	Sí		Orden de esa fila de nulas.
created_at	timestamp	Sí		Marca de tiempo de Eloquent.
updated_at	timestamp	Sí		Marca de tiempo de Eloquent.

Tabla compras

Facturas de compra al M.H.

Columna	Tipo	Nulo	Clave	Descripción
id	bigint(20) unsigned	No	PK	Identificador autoincremental.
fecha	date	No		Fecha de la compra; ingreso a bodega.
numero_factura	varchar(50)	No		Factura del M.H. (única, validada en la aplicación).
monto_total	decimal(12,2)	Sí		Σ cantidad × valor de los lotes (recalculado).
observaciones	text	Sí		Notas opcionales.
user_id	bigint(20) unsigned	Sí	FK/IDX	Usuario que registró la compra.

Columna	Tipo	Nulo	Clave	Descripción
created_at	timestamp	Sí		Marca de tiempo de Eloquent.
updated_at	timestamp	Sí		Marca de tiempo de Eloquent.

Tabla lotes

Lotes de documentos de cada compra.

Columna	Tipo	Nulo	Clave	Descripción
id	bigint(20) unsigned	No	PK	Identificador autoincremental.
compra_id	bigint(20) unsigned	No	FK/IDX	Compra a la que pertenece.
tipo_especie_id	bigint(20) unsigned	No	FK/IDX	Clave foránea.
denominacion_id	bigint(20) unsigned	No	FK/IDX	Clave foránea.
serie	varchar(10)	Sí		Serie normalizada (mayúsculas; null si no tiene).
cantidad_total	int(10) unsigned	No		Σ documentos de sus rangos.
created_at	timestamp	Sí		Marca de tiempo de Eloquent.
updated_at	timestamp	Sí		Marca de tiempo de Eloquent.

Tabla lote_rangos

Rangos de numeración de cada lote.

Columna	Tipo	Nulo	Clave	Descripción
id	bigint(20) unsigned	No	PK	Identificador autoincremental.
lote_id	bigint(20) unsigned	No	FK/IDX	Clave foránea.
numero_inicio	int(10) unsigned	No		Primer número del rango (inclusive).
numero_fin	int(10) unsigned	No		Último número del rango (inclusive).

Tabla traslados

Encabezado de cada movimiento entre bodega y distritos.

Columna	Tipo	Nulo	Clave	Descripción
id	bigint(20) unsigned	No	PK	Identificador autoincremental.
tipo	varchar(20)	No		bodega_distrito, distrito_bodega o distrito_distrito.
distrito_id	bigint(20) unsigned	Sí	FK/IDX	Destino (null en devoluciones a bodega).
origen_distrito_id	bigint(20) unsigned	Sí	FK/IDX	Origen (null cuando sale de bodega).
fecha	date	No		Fecha del movimiento.
observaciones	text	Sí		Notas opcionales.

Columna	Tipo	Nulo	Clave	Descripción
usuario_id	bigint(20) unsigned	No	FK/IDX	Usuario que registró.
created_at	timestamp	Sí		Marca de tiempo de Eloquent.
updated_at	timestamp	Sí		Marca de tiempo de Eloquent.

Tabla traslado_detalles

Rangos movidos en cada traslado.

Columna	Tipo	Nulo	Clave	Descripción
id	bigint(20) unsigned	No	PK	Identificador autoincremental.
traslado_id	bigint(20) unsigned	No	FK/IDX	Traslado al que pertenece.
lote_id	bigint(20) unsigned	No	FK/IDX	Lote del que se mueve el rango.
numero_inicio	int(10) unsigned	No		Primer número del rango (inclusive).
numero_fin	int(10) unsigned	No		Último número del rango (inclusive).
cantidad	int(10) unsigned	No		Documentos del rango.

Tabla nulas

Rangos anulados en los distritos.

Columna	Tipo	Nulo	Clave	Descripción
id	bigint(20) unsigned	No	PK	Identificador autoincremental.
traslado_detalle_id	bigint(20) unsigned	No	FK/IDX	Detalle por el que llegó el rango al distrito.
distrito_id	bigint(20) unsigned	No	FK/IDX	Distrito que anula.
numero_inicio	int(10) unsigned	No		Primer número del rango (inclusive).
numero_fin	int(10) unsigned	No		Último número del rango (inclusive).
fecha	date	No		Fecha de la anulación.
motivo	varchar(255)	Sí		Motivo opcional.
usuario_id	bigint(20) unsigned	No	FK/IDX	Usuario que registró.
created_at	timestamp	Sí		Marca de tiempo de Eloquent.
updated_at	timestamp	Sí		Marca de tiempo de Eloquent.

Tabla realizaciones

Rangos realizados (vendidos/entregados) en los distritos.

Columna	Tipo	Nulo	Clave	Descripción
id	bigint(20) unsigned	No	PK	Identificador autoincremental.

Columna	Tipo	Nulo	Clave	Descripción
tipo_especie_id	bigint(20) unsigned	No	FK/IDX	Clave foránea.
denominacion_id	bigint(20) unsigned	No	FK/IDX	Clave foránea.
serie	varchar(10)	Sí		Serie de los documentos realizados.
distrito_id	bigint(20) unsigned	No	FK/IDX	Distrito que realiza.
numero_inicio	int(10) unsigned	No		Primer número del rango (inclusive).
numero_fin	int(10) unsigned	No		Último número del rango (inclusive).
cantidad	int(10) unsigned	No		numero_fin – numero_inicio + 1.
fecha	date	No		Fecha de la realización.
monto_cobrado	decimal(12,2)	No		cantidad × precio de venta del distrito (2 decimales).
usuario_id	bigint(20) unsigned	No	FK/IDX	Usuario que registró.
created_at	timestamp	Sí		Marca de tiempo de Eloquent.
updated_at	timestamp	Sí		Marca de tiempo de Eloquent.

Tabla users

Usuarios del sistema.

Columna	Tipo	Nulo	Clave	Descripción
id	bigint(20) unsigned	No	PK	Identificador autoincremental.
usuario	varchar(255)	No	UNI	Nombre de inicio de sesión (único).
password	varchar(255)	No		Hash bcrypt (12 rondas).
rol	varchar(255)	No		Copia del rol principal (el control real lo hace Spatie).
remember_token	varchar(100)	Sí		Token de «recordarme».
created_at	timestamp	Sí		Marca de tiempo de Eloquent.
updated_at	timestamp	Sí		Marca de tiempo de Eloquent.

Tabla anios

Periodos de las constancias de retención.

Columna	Tipo	Nulo	Clave	Descripción
id	bigint(20) unsigned	No	PK	Identificador autoincremental.
anio	year(4)	No	UNI	Año fiscal (único).
archivo_excel	varchar(255)	Sí		Nombre del archivo en public/excel con los datos de retención del año.
created_at	timestamp	Sí		Marca de tiempo de Eloquent.

Columna	Tipo	Nulo	Clave	Descripción
updated_at	timestamp	Sí		Marca de tiempo de Eloquent.

Tabla consultas

Bitácora de búsquedas públicas de constancias.

Columna	Tipo	Nulo	Clave	Descripción
id	bigint(20) unsigned	No	PK	Identificador autoincremental.
nitdui	varchar(255)	No		NIT/DUI buscado tal como se escribió.
nombre	varchar(255)	Sí		Nombre encontrado (null si no hubo coincidencia).
buscado_en	timestamp	No		Fecha y hora de la búsqueda.

Tablas de Spatie Permission: `roles` y `permissions` (`name`, `guard_name`, únicos en conjunto), `model_has_roles`, `model_has_permissions` y `role_has_permissions` (tablas pivote con clave primaria compuesta). El sistema usa solo roles; no hay permisos definidos.

Tablas de infraestructura: `sessions` (sesiones de usuario, índices en `user_id` y `last_activity`), `cache` y `cache_locks` (almacén de caché, incluidos los PDF temporales y los contadores de límite de intentos), `jobs`, `job_batches` y `failed_jobs` (cola de base de datos, sin trabajos en uso) y `migrations`.

5.5 Claves foráneas

Tabla	Columna	Referencia	ON DELETE
compras	user_id	users	RESTRICT
denominaciones	tipo_especie_id	tipo_especies	RESTRICT
distrito_denominacion	distrito_id / denominacion_id	distritos / denominaciones	RESTRICT
distrito_tipo_especie	distrito_id / tipo_especie_id	distritos / tipo_especies	RESTRICT
lotes	compra_id / tipo_especie_id / denominacion_id	compras / tipo_especies / denominaciones	RESTRICT
lote_rangos	lote_id	lotes	RESTRICT
traslados	distrito_id / origen_distrito_id / usuario_id	distritos / distritos / users	RESTRICT
traslado_detalles	traslado_id / lote_id	traslados / lotes	RESTRICT
nulas	traslado_detalle_id / distrito_id / usuario_id	traslado_detalles / distritos / users	RESTRICT
realizaciones	tipo_especie_id / denominacion_id / distrito_id / usuario_id	tipo_especies / denominaciones / distritos / users	RESTRICT
model_has_roles, model_has_permissions, role_has_permissions	role_id / permission_id	roles / permissions	CASCADE

Todas las claves de negocio usan `RESTRICT`: la base impide borrar un catálogo, distrito, usuario o lote que tenga movimientos. Los controladores eliminan explícitamente los hijos (rangos de un lote, detalles de un traslado) dentro de la misma transacción y antes comprueban las reglas de negocio.

5.6 Índices

Tabla	Índice	Tipo	Columnas
anios	anios_anio_unique	Único	anio
distritos	distritos_codigo_unique	Único	codigo
users	users_usuario_unique	Único	usuario
distrito_denominacion	…_distrito_id_denominacion_id_unique	Único	distrito_id, denominacion_id
distrito_tipo_especie	…_distrito_id_tipo_especie_id_unique	Único	distrito_id, tipo_especie_id
realizaciones	realizaciones_tipo_especie_id_serie_numero_inicio_index	Compuesto	tipo_especie_id, serie, numero_inicio
roles / permissions	…_name_guard_name_unique	Único	name, guard_name
sessions	sessions_user_id_index, sessions_last_activity_index	Simple	user_id; last_activity
cache / cache_locks	…_expiration_index	Simple	expiration
Todas las FK	<tabla>_<columna>_foreign	Simple	Columna de la clave foránea

Restricciones que se garantizan en la aplicación (no en la base): unicidad de `numero_factura`; inicio ≤ fin en todos los rangos; no solapamiento de rangos entre lotes del mismo tipo y serie; un número se realiza una sola vez y se anula una sola vez; no se anula lo realizado ni se realiza lo anulado; y la validez temporal de la posesión (sección 4.3).

5.7 Migraciones

Migración	Propósito
0001_01_01_000000_create_users_table	`users` (con `usuario` y `rol` en lugar de email) y `sessions`.
0001_01_01_000001_create_cache_table	`cache` y `cache_locks`.
0001_01_01_000002_create_jobs_table	`jobs`, `job_batches`, `failed_jobs`.
2026_05_10_014755_create_permission_tables	Tablas de Spatie Permission.
2026_05_10_020000_create_anios_table	Años de retención.
2026_05_20_032705_add_archivo_excel_to_anios_table	Archivo Excel por año.
2026_06_02_000000_create_consultas_table	Bitácora de búsquedas de constancias.
2026_07_07_000001 … 000010	Primera versión de las tablas de especies (distritos, tipos, denominaciones, compras, lotes, rangos, nulas, traslados, detalles, realizaciones).
2026_07_09_000001_rebuild_especies_municipales	Reconstruye las tablas de especies: las nulas pasan a referenciar `traslado_detalles` y se agrega `distrito_id`.
2026_07_21_221627_add_user_id_to_compras_table	Usuario que registra la compra.

Migración	Propósito
2026_08_06_153502_rebuild_realizaciones_as_ranges	Realizaciones como rangos (`numero_inicio`, `numero_fin`, `cantidad`).
2026_09_06_000001_add_tipo_to_traslados_table	`tipo` y `origen_distrito_id`: devoluciones y traslados entre distritos.
2026_09_24_000001_add_descripcion_precio_costo_to_denominaciones_table	Descripción y precio de costo (4 decimales).
2026_09_29_000001_add_serie_to_realizaciones_table	Serie en realizaciones e índice compuesto.
2026_09_30_000001_add_configuracion_reportes_a_especies	`unidades` y `orden` en tipos, `precio_venta`, datos de distrito, `distrito_tipo_especie` y `distrito_denominacion`.
2026_10_19_000001_add_realizaciones_a_configuracion_distritos	Título, rótulos, orden y filas de nulas del reporte Especies Realizadas.
2026_10_25_000001_drop_nombre_contribuyente_from_realizaciones	Elimina el campo nombre del contribuyente.
2026_10_25_000002_drop_cajas_de_distrito	Elimina `ubicaciones` y `ubicacion_rangos` (cajas de distrito sin uso); `down()` las recrea.

Las 27 migraciones están aplicadas en la base actual. Las de reconstrucción (`rebuild_*`) borran y recrean tablas: solo deben ejecutarse en orden sobre una base nueva, nunca de forma aislada sobre datos productivos.

5.8 Seeders y volumen de datos

Seeder	Contenido
RolesSeeder	Roles `admin`, `empleado`, `visitante` (idempotente con `firstOrCreate`).
EspeciesMunicipalesSeeder	Distritos, tipos de especie y denominaciones de Fondo Vialidad de la instalación inicial.
DenominacionesSeeder	Denominaciones del resto de tipos (Carnet de Menores inactivo).
CatalogoEspeciesSeeder	Distrito D05, unidades y orden de tipos, costos y precios, títulos y tarifas por distrito, plantillas de los reportes. Se puede ejecutar varias veces (actualiza).
DatabaseSeeder	Ejecuta los anteriores, crea los usuarios `admin` y `empleado` y llama a DatosInicialesEspeciesSeeder.
DatosInicialesEspeciesSeeder	Movimientos de noviembre de 2025 a junio de 2026 desde el CSV (sección 4.8).

Volumen actual (25/10/2026): 5 distritos, 17 tipos, 27 denominaciones, 37 configuraciones de tipo, 46 tarifas, 11 compras, 117 lotes y rangos, 19 traslados con 120 detalles, 1 028 realizaciones y 769 nulas (30/11/2025 a 30/06/2026), 3 usuarios, 2 años y 2 consultas.

6. APIs, Endpoints e Integraciones Externas

6.1 Estilo de interfaz

El sistema **no expone una API pública** (REST, GraphQL ni similar) ni usa autenticación por token. Todas las rutas están en `routes/web.php`, pasan por el grupo de middleware `web` (sesión, cookies cifradas, protección CSRF) y devuelven vistas HTML, redirecciones, PDF o, en cinco casos, JSON para formularios dinámicos. `php artisan route:list --except-vendor` reporta 85 rutas; Laravel registra además la ruta de salud `/up`.

Convención	Comportamiento
Formularios	POST con `@csrf`; PATCH y DELETE con `@method`. Sin token válido Laravel responde 419 (vista `errors/419` personalizada).
Respuestas de escritura	Redirección con mensajes flash (`success`, `error`, `success_lote`, `error_detalle`, …) y errores de validación en `$errors`, conservando la entrada con `withInput()`.
Sin sesión	El middleware `auth` redirige a `/login`.
Rol insuficiente	El middleware `role:` de Spatie lanza `UnauthorizedException` → 403 (vista `errors/403`).
Modelo inexistente	Route model binding → 404 (vista `errors/404`).
Páginas autenticadas	Middleware `no-back`: cabeceras `Cache-Control: no-cache, no-store, max-age=0, must-revalidate`, `Pragma: no-cache` y `Expires` en el pasado, para que el botón «atrás» no muestre páginas tras cerrar sesión.

6.2 Rutas públicas y de invitados

Método	URI	Nombre	Acción / Middleware
GET	/	—	Redirige a `retencion`.
GET	retencion	retencion	`RetencionController@index`: formulario y visor del PDF.
POST	retencion/buscar	retencion.buscar	`RetencionController@buscar` · `throttle:retencion` (20/min por IP para invitados).
GET	retencion/pdf/{token}	retencion.pdf.ver	`RetencionController@verPdf`: PDF inline desde caché (404 si expiró).
GET	login	login	`LoginController@showLoginForm` · `guest`.
POST	login	—	`LoginController@login` · `guest` · límite 10/min por usuario+IP y 30/min por IP.
GET	logout	logout	`LoginController@logout`: invalida la sesión y regenera el token CSRF.
GET	home	home	`DashboardController@redirect` · `auth`.

6.3 Rutas de administración (role:admin)

Método	URI	Acción
GET	admin/roles	`RolController@index`

Método	URI	Acción
PATCH	admin/roles/{user}	`RolController@update` (rol ∈ admin, empleado)
GET	admin/usuarios · admin/usuarios/create · admin/usuarios/{usuario}/edit	`UsuarioController@index|create|edit`
POST · PUT/PATCH · DELETE	admin/usuarios · admin/usuarios/{usuario}	`UsuarioController@store|update|destroy`

6.4 Rutas compartidas (role:admin|empleado, prefijo admin)

Módulo	Método y URI	Acción
Panel	GET admin/dashboard	`DashboardController@admin`
Años	GET/POST admin/anios · GET admin/anios/{anio}/edit · PATCH/DELETE admin/anios/{anio}	`AnioController`
Consultas	GET admin/consultas · POST admin/consultas/generar · GET admin/consultas/pdf/{token}	`ConsultaController@index|generarPdf|verPdf`
Tipos	GET/POST admin/especies/configuracion/tipos · PATCH/DELETE …/tipos/{tipo}	`ConfiguracionController`
Denominaciones	GET/POST admin/especies/configuracion/denominaciones · PATCH/DELETE …/{denominacion}	`ConfiguracionController`
Distritos	GET admin/especies/configuracion/distritos · PATCH …/distritos/{distrito} · POST …/{distrito}/tipos · POST …/{distrito}/tarifas · DELETE …/tipos/{config} · DELETE …/tarifas/{tarifa}	`DistritoConfigController`
Compras	GET admin/especies/compras · GET …/compras/crear · POST …/compras · GET/PATCH/DELETE …/compras/{compra} · GET …/{compra}/lotes/crear · POST …/{compra}/lotes · DELETE …/{compra}/lotes/{lote}	`CompraController`
Traslados	GET admin/especies/bodega/traslados · GET …/traslados/crear · POST …/traslados · GET/PATCH/DELETE …/traslados/{traslado} · GET …/{traslado}/detalles/crear · POST …/{traslado}/detalles · DELETE …/{traslado}/detalles/{detalle}	`BodegaController`
Stock	GET admin/especies/bodega/stock · GET admin/especies/distritos/stock	`BodegaController@stock`, `DistritoController@stock`
Anulaciones	GET admin/especies/distritos/anulaciones · GET …/crear · POST … · GET …/{nula}/editar · PATCH/DELETE …/{nula}	`DistritoController`
Realizaciones	GET admin/especies/realizaciones · GET …/crear · POST … · GET …/{realizacion}/editar · PATCH/DELETE …/{realizacion}	`RealizacionController`
Reportes	GET admin/especies/reportes/{reporte}, donde reporte = libro, realizaciones, saldos, bodega, distritos, traslados, mensual o anual	`ReporteController` (formulario, o PDF si viene `generar`)

La ruta `GET empleado/dashboard` (`DashboardController@empleado`) corresponde al rol `visitante`. Todas las rutas de los grupos protegidos llevan además el middleware `no-back`.

6.5 Endpoints AJAX (JSON)

Son GET internos, protegidos por sesión y rol como el resto del grupo compartido; los consume jQuery (`$.getJSON`) en los formularios.

Endpoint	Parámetros	Respuesta
admin/especies/ajax/denominaciones	`tipo_especie_id`	Arreglo de denominaciones activas: `[{id, descripcion, valor, precio_costo, precio_venta}]`.
admin/especies/ajax/lotes-stock	`tipo_especie_id`	Lotes con existencia en bodega: `[{id, label, disponible, rangos:[{inicio, fin}]}]`.
admin/especies/ajax/lotes-distrito-stock	`distrito_id`, `tipo_especie_id`	Lotes con existencia en el distrito (mismo formato), para devoluciones y traslados entre distritos.
admin/especies/ajax/detalles-disponibles	`distrito_id`, `tipo_especie_id`, `nula_id` (opcional, al editar)	`[{id, label, inicio, fin, disponible, rangos:[{inicio, fin}], ya_anulados:[{inicio, fin}]}]` solo con detalles que conservan existencia.
admin/especies/ajax/realizacion-info	`distrito_id`, `tipo_especie_id`, `ignorar` (opcional, id de la realización en edición)	`{recibido, salido, anulado, realizado, disponible, grupos:[{denominacion_id, valor, precio_venta, etiqueta, serie, disponible, rangos}]}`.

```
GET /admin/especies/ajax/realizacion-info?distrito_id=1&tipo_especie_id=1
{
  "recibido": 30000, "salido": 0, "anulado": 12, "realizado": 21950, "disponible": 8038,
  "grupos": [
    { "denominacion_id": 1, "valor": 0.57, "precio_venta": 0.57, "etiqueta": "$0.57",
      "serie": "C", "disponible": 1250, "rangos": [ { "inicio": 52751, "fin": 54000 } ] }
  ]
}
```

> *Los valores del ejemplo son ilustrativos del formato.*

6.6 Generación y entrega de PDF

- **Reportes de especies:** `GET …/reportes/<reporte>?generar=1&…` responde directamente `application/pdf` con `stream()` (visualización en el navegador, nombre como `libro-D01-5-2026.pdf`).
- **Constancias y reporte de consultas:** el POST genera el PDF, lo guarda en caché (base64, 30 minutos) bajo un UUID v4 y redirige; la vista lo carga en un iframe desde `…/pdf/{token}` con `Content-Disposition: inline`.

6.7 Integraciones y dependencias externas

Integración	Tipo	Uso	Observaciones
mPDF (vía carlos-meneses/laravel-mpdf)	Librería local	Render de HTML/CSS a PDF.	Sin servicio externo; usa directorio temporal en disco.
PhpSpreadsheet	Librería local	Lectura de los Excel de retención.	Carga el libro completo en memoria en cada búsqueda.

Integración	Tipo	Uso	Observaciones
Archivos Excel de retención	Archivos en `public/excel`	Fuente de datos de constancias.	Se registran por año en el módulo Años. Ver riesgo en 7.4.
SweetAlert2	CDN jsDelivr (`cdn.jsdelivr.net/npm/sweetalert2@11`)	Diálogos de confirmación.	Requiere internet en el navegador; sin hash SRI.
Google Fonts	CDN de Google (AdminLTE, `google_fonts.allowed = true`)	Tipografía Source Sans Pro.	Si no hay internet se usa la fuente de respaldo.
AdminLTE, Bootstrap 4, jQuery, Font Awesome, OverlayScrollbars	Activos locales en `public/vendor`	Interfaz.	Publicados por el paquete AdminLTE.

No hay integraciones con servicios de terceros: no se envían correos (`MAIL_MAILER=log`), no hay pasarelas de pago, OAuth, almacenamiento en la nube ni colas de trabajo (la cola está configurada en base de datos pero no se despacha ningún job).

7. Configuración, Variables de Entorno y Seguridad

7.1 Variables de entorno

Valores del entorno local actual (los secretos se omiten) y recomendación para producción:

Variable	Valor local	Descripción	Producción
APP_NAME	"Sistema de Especies Municipales"	Nombre de la aplicación.	Igual.
APP_ENV	local	Entorno.	`production`
APP_KEY	(definida)	Clave de cifrado de cookies y sesión.	Generar con `key:generate`; nunca versionar.
APP_DEBUG	true	Trazas detalladas de error.	`false`
APP_URL	http://localhost	URL base.	URL pública con https.
APP_LOCALE / APP_FALLBACK_LOCALE	en / en	Idioma de Laravel (los mensajes de validación están personalizados en español).	Opcional `es`.
BCRYPT_ROUNDS	12	Costo de bcrypt.	12 o más.
LOG_CHANNEL / LOG_STACK / LOG_LEVEL	stack / single / debug	Bitácora en `storage/logs/laravel.log`.	`daily` y `warning`.
DB_CONNECTION	mysql	Driver (MariaDB).	`mysql`
DB_HOST / DB_PORT / DB_DATABASE	127.0.0.1 / 3306 / db_proyecto_renta	Conexión a la base.	Según servidor.
DB_USERNAME / DB_PASSWORD	root / (vacía en Laragon)	Credenciales.	Usuario dedicado con privilegios mínimos.
SESSION_DRIVER / SESSION_LIFETIME	database / 120	Sesiones en tabla `sessions`, 120 minutos.	Igual; añadir `SESSION_SECURE_COOKIE=true`.
SESSION_ENCRYPT	false	Cifrado del contenido de la sesión.	Opcional `true`.
CACHE_STORE	database	Caché en tabla `cache` (PDF temporales, rate limiter).	Igual o `redis`.
QUEUE_CONNECTION	database	Cola (sin uso actual).	Igual.
MAIL_MAILER	log	Correo (sin uso).	Igual.
FILESYSTEM_DISK	local	Disco por defecto.	Igual.
ESPECIES_FIRMA_TESORERIA	(no definida; usa el valor por defecto)	Nombre de la tesorera en las firmas de los PDF.	Definir al cambiar de titular.
REDIS_*, MEMCACHED_HOST, AWS_*	Valores de plantilla	No se usan.	—

> *Hallazgo: `.env.example` conserva `DB_CONNECTION=sqlite` de la plantilla de Laravel, mientras que el sistema requiere MariaDB/MySQL (migraciones de reconstrucción y `lockForUpdate`). Debe actualizarse con las variables `DB_*` reales y `ESPECIES_FIRMA_TESORERIA`.*

7.2 Archivos de configuración propios

- `config/especies.php`: códigos de formulario (`libro` TESO-008-FORM, `bodega` TESO-010-FORM, `realizaciones` TESO-002-CONT, `saldos` TESO-004-CONT, `otros` TESO-000-FORM), versiones (001, 000, 001, 000), vigencia 11/11/2025 y firma de la tesorera municipal.
- `config/adminlte.php`: título, logo «Sistema de Especies Municipales» centrado, barra lateral fija (`layout_fixed_sidebar`), menú con atributos `can` que se evalúan contra los Gates, y SweetAlert2 como plugin.
- `config/permission.php`: configuración por defecto de Spatie (guard `web`, caché de permisos de 24 horas).
- `config/session.php`, `cache.php`, `queue.php`: drivers de base de datos tomados de `.env`.

7.3 Controles de seguridad implementados

Amenaza	Control	Implementación
Contraseñas expuestas	Hash bcrypt (12 rondas)	Cast `password => hashed` en `User`.
Fuerza bruta en el login	Límite de intentos	10 fallos por usuario+IP y 30 por IP por minuto (`RateLimiter` en `LoginController`); mensaje genérico «Credenciales incorrectas».
Fijación de sesión	Regeneración de sesión	`session()->regenerate()` al iniciar; `invalidate()` y `regenerateToken()` al cerrar.
CSRF	Token por formulario	Middleware `web` + `@csrf`.
XSS	Escape de salida	Blade `{{ }}` escapa por defecto en todas las vistas.
Inyección SQL	Consultas parametrizadas	Eloquent y Query Builder; los `selectRaw` no interpolan datos del usuario.
Acceso no autorizado	RBAC	Middleware `auth` + `role:admin` / `role:admin|empleado`; Gates para el menú; rol desconocido → cierre de sesión.
Enumeración de NIT/DUI	Límite por IP	`throttle:retencion`: 20 búsquedas por minuto por IP para invitados.
Exposición de PDF	Token temporal	UUID v4 (122 bits aleatorios) en caché con expiración de 30 minutos.
Páginas en caché del navegador	Cabeceras no-store	Middleware `no-back`.
Condiciones de carrera en inventario	Bloqueo pesimista + transacción	`Inventario::bloquear` + `DB::transaction` + `asegurar` con reversión.
Datos inconsistentes	Validación de dominio	Reglas de rango, no solapamiento y verificación temporal (sección 4.3).
Reenvío de formularios	Post/Redirect/Get	Todas las escrituras redirigen.
Redirección abierta	Lista blanca	El parámetro `volver` al editar una anulación solo se acepta si empieza por la URL del historial.

7.4 Hallazgos de la auditoría y recomendaciones

#	Severidad	Hallazgo	Recomendación
1	Alta	Los Excel de retención en `public/excel` (nombres, NIT/DUI, salarios y retenciones) son descargables por URL directa sin autenticación y además están versionados en el repositorio (que es público).	Moverlos a `storage/app/private`, leerlos solo desde el controlador, retirarlos del repositorio y purgar el historial de Git.
2	Alta	`database/data/db_proyecto_renta.sql` y `old_db_proyecto_renta.sql` están versionados: contienen hashes de contraseñas, sesiones y datos completos.	Retirarlos del repositorio, purgar el historial, agregarlos a `.gitignore` y rotar las contraseñas.
3	Alta (al desplegar)	`DatabaseSeeder` crea `admin/admin123` y `empleado/empleado123`.	Cambiar las contraseñas tras la instalación o leerlas de variables de entorno.
4	Media	`APP_DEBUG=true`, `APP_ENV=local` y `LOG_LEVEL=debug` en el `.env` actual.	Valores de producción de la tabla 7.1.
5	Media	Cierre de sesión por `GET /logout`: un enlace o imagen de un tercero puede cerrar la sesión (CSRF de logout).	Cambiar a `POST` con token CSRF.
6	Media	Contraseña mínima de 6 caracteres sin requisitos de complejidad.	`Password::min(10)->letters()->numbers()`.
7	Media	Sin pruebas automatizadas propias: `tests/` solo contiene los ejemplos de Laravel. Las validaciones se hicieron con scripts externos al repositorio.	Pruebas de característica para `Inventario` (neto, verificar), flujos de compra/traslado/realización/nula y cuadre de reportes.
8	Media	Error funcional: en el Reporte Mensual con «Todos los distritos», `pdfMensual()` reutiliza la variable `$distrito` en un `foreach`, y el encabezado muestra el último distrito (D05) en lugar de «TODOS LOS DISTRITOS». Las filas de datos son correctas.	Renombrar la variable del ciclo.
9	Media	No hay bitácora de auditoría de ediciones y eliminaciones (realizaciones, nulas, traslados): solo se guarda el usuario que creó el registro.	Tabla de auditoría o paquete de activity log.
10	Baja	`.env.example` usa SQLite (ver 7.1).	Actualizar la plantilla.
11	Baja	El rol se guarda dos veces: columna `users.rol` y tablas de Spatie. `RolController` actualiza ambas, pero cualquier cambio directo puede desincronizarlas; el seeder de datos busca al admin por `users.rol`.	Usar solo Spatie (`User::role('admin')`).
12	Baja	Cinco vistas Blade sin ruta (sección 3) y dependencias Vite/Tailwind sin uso real.	Eliminar para reducir superficie y mantenimiento.
13	Baja	SweetAlert2 y Google Fonts se cargan desde CDN sin SRI; el sistema depende de internet para esos recursos.	Servirlos localmente desde `public/vendor`.
14	Baja	La tabla `consultas` guarda cada NIT/DUI buscado sin política de retención.	Definir plazo de conservación y depuración periódica.
15	Informativo	El inventario se calcula en línea y `verificar()` recorre la historia completa del lote.	Monitorear tiempos; considerar cierres mensuales si el volumen crece varios órdenes de magnitud.

8. Guía de Despliegue (Deployment) y Entorno Local

8.1 Requisitos

Componente	Versión	Notas
PHP	≥ 8.3 (probado con 8.4.12)	Extensiones: `pdo_mysql`, `mbstring`, `gd`, `zip`, `xml`, `dom`, `fileinfo`, `openssl`, `bcmath`, `intl`, `ctype`, `tokenizer`.
Composer	2.x	Instalación de dependencias PHP.
MariaDB / MySQL	MariaDB ≥ 10.6 o MySQL 8 (probado con MariaDB 12.1.2)	InnoDB, `utf8mb4_unicode_ci`.
Servidor web	Apache 2.4 o Nginx	Raíz del documento en `public/`.
Node.js	20+ (opcional)	Solo para compilar los activos Vite de la página de bienvenida; la interfaz no los necesita.

8.2 Entorno local con Laragon

1. Clonar o copiar el proyecto en `C:\laragon\www\ProyectoServicioSocial`. Laragon crea el host virtual `http://proyectoserviciosocial.test`.
2. Instalar dependencias y preparar el entorno:

```
composer install
copy .env.example .env          (en Git Bash: cp .env.example .env)
php artisan key:generate
```

3. Editar `.env`: `DB_CONNECTION=mysql`, `DB_HOST=127.0.0.1`, `DB_PORT=3306`, `DB_DATABASE=db_proyecto_renta`, `DB_USERNAME=root`, `DB_PASSWORD=` (vacía en Laragon).
4. Crear la base y ejecutar migraciones y seeders (crea el esquema, el catálogo, los usuarios y carga noviembre 2025–junio 2026):

```
mysql -u root -e "CREATE DATABASE db_proyecto_renta CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci"
php artisan migrate --seed
```

5. Copiar los archivos Excel de retención a `public/excel` y registrarlos en el módulo Años.
6. Abrir `http://proyectoserviciosocial.test` (o ejecutar `php artisan serve` y abrir `http://127.0.0.1:8000`), iniciar sesión con `admin` y cambiar la contraseña.

Script alternativo: `composer run dev` levanta en paralelo el servidor, el listener de cola, `pail` (logs) y Vite; no es necesario para el uso normal.

Recargar los datos iniciales

`migrate:fresh` **borra todas las tablas**. Respaldar primero:

```
mysqldump -u root --routines db_proyecto_renta > respaldo_AAAAMMDD.sql
php artisan migrate:fresh --seed
```

`DatosInicialesEspeciesSeeder` no se ejecuta si ya existen compras, traslados, realizaciones o nulas, para evitar duplicados.

8.3 Despliegue en producción

1. Servidor con PHP-FPM y Nginx/Apache, HTTPS obligatorio y raíz del sitio en `public/` (nunca en la raíz del proyecto).
2. Instalar dependencias sin paquetes de desarrollo: `composer install --no-dev --optimize-autoloader`.
3. Configurar `.env` con los valores de producción (sección 7.1) y un usuario de base de datos dedicado.
4. Primera instalación: `php artisan key:generate`, `php artisan migrate --force` y `php artisan db:seed --force`; cambiar de inmediato las contraseñas sembradas.
5. Optimizar: `php artisan optimize` (cachea configuración, rutas, vistas y eventos).
6. Permisos de escritura para el usuario del servidor web en `storage/` y `bootstrap/cache/` (mPDF escribe archivos temporales).
7. Antes de publicar, resolver los hallazgos 1 a 3 de la sección 7.4 (archivos Excel y volcados SQL fuera del alcance público, contraseñas por defecto).

No se requieren tareas programadas (`schedule:run`) ni un worker de colas, porque el sistema no define jobs ni comandos programados.

Actualizaciones

```
php artisan down
git pull
composer install --no-dev --optimize-autoloader
php artisan migrate --force
php artisan optimize
php artisan up
```

Respaldos

- Volcado diario de la base con `mysqldump` (incluye la configuración de reportes y todos los movimientos).
- Copia de `public/excel` (o de su nueva ubicación privada) y de `.env`.
- Probar periódicamente la restauración en un entorno aparte.

8.4 Docker

El repositorio **no incluye** `Dockerfile`, `docker-compose.yml` ni Laravel Sail: el entorno de referencia es Laragon en Windows. Si se desea contenerizar, se necesitan tres servicios: aplicación (PHP-FPM 8.4 con las extensiones de 8.1), servidor web (Nginx apuntando a `public/`) y base de datos (MariaDB con volumen persistente), más volúmenes para `storage/` y para los archivos Excel de retención.

8.5 Verificación posterior al despliegue

- Iniciar sesión como admin y como empleado y comprobar el menú de cada rol.
- Generar el Libro, Especies Realizadas y Control de Saldos de un distrito y un mes con datos (por ejemplo, junio de 2026).
- Buscar una constancia de retención con un NIT/DUI conocido.
- Revisar `storage/logs/laravel.log` y `php artisan route:list`.

8.6 Problemas frecuentes

Síntoma	Causa probable	Solución
Página 419 «Page Expired»	Sesión expirada o token CSRF inválido.	Recargar la página e iniciar sesión de nuevo.
«Demasiados intentos fallidos»	Límite de inicio de sesión alcanzado.	Esperar el tiempo indicado (1 minuto).
«Demasiadas búsquedas desde esta conexión»	Límite de 20 búsquedas por minuto en constancias.	Esperar un minuto.
«No se encontró el archivo Excel del año seleccionado»	El archivo registrado en Años no está en `public/excel`.	Copiar el archivo o corregir el registro del año.
Error al generar PDF	Sin permisos de escritura para los temporales de mPDF.	Dar permisos a `storage/` y al directorio temporal del sistema.
«Ya hay movimientos de especies registrados» al sembrar	El seeder de datos iniciales protege contra duplicados.	Usar `migrate:fresh --seed` (previo respaldo) si se quiere recargar.
Mensaje «El distrito … no tenía disponibles …» al guardar	La operación rompería la historia del inventario.	Revisar fechas y rangos de los movimientos anteriores indicados en el mensaje.